

Часть I. Библиотека STL

Что такое STL

STL (Standard Template Library) — это набор обобщённых компонентов, входящий в стандарт C++ с 1998 года. Библиотека построена на парадигме обобщённого программирования: её компоненты работают с любыми типами данных, удовлетворяющими определённым требованиям. STL состоит из пяти основных частей: контейнеры, итераторы, алгоритмы, функторы и адаптеры.

Контейнеры

Контейнеры — это структуры данных для хранения объектов. Они делятся на несколько категорий.

Последовательные контейнеры хранят элементы в порядке их добавления:

- `std::vector` — динамический массив с непрерывным хранением;
- `std::deque` — двусторонняя очередь с блочным хранением;
- `std::list` — двунаправленный связный список;
- `std::forward_list` — однонаправленный связный список;
- `std::array` — статический массив фиксированного размера.

Ассоциативные контейнеры хранят элементы в отсортированном порядке и обеспечивают быстрый поиск:

- `std::set` — множество уникальных элементов;
- `std::multiset` — множество с повторениями;
- `std::map` — словарь «ключ-значение» с уникальными ключами;
- `std::multimap` — словарь с повторяющимися ключами.

Неупорядоченные ассоциативные контейнеры используют хеш-таблицы:

- `std::unordered_set`, `std::unordered_multiset`;
- `std::unordered_map`, `std::unordered_multimap`.

Адаптеры контейнеров изменяют интерфейс других контейнеров:

- `std::stack` — стек (LIFO);
- `std::queue` — очередь (FIFO);
- `std::priority_queue` — приоритетная очередь.

Все контейнеры поддерживают единообразный интерфейс: `begin`, `end`, `size`, `empty`, `insert`, `erase`, `clear`. Это позволяет писать обобщённый код, работающий с любым контейнером.

Итераторы

Итераторы — это обобщённые указатели, обеспечивающие доступ к элементам контейнера. Они играют роль связующего звена между контейнерами и алгоритмами. Итераторы делятся на пять категорий:

1. Входные (input) — только чтение, только вперёд.
2. Выходные (output) — только запись, только вперёд.
3. Однонаправленные (forward) — чтение и запись, только вперёд.
4. Двухнаправленные (bidirectional) — чтение и запись, вперёд и назад.
5. Произвольного доступа (random access) — чтение и запись, прыжки на n позиций за $O(1)$.

Каждый контейнер предоставляет итератор своей категории. Например, `std::vector` даёт итераторы произвольного доступа, а `std::list` — двухнаправленные.

Основные операции с итераторами:

```
auto it = container.begin(); // итератор на первый элемент
auto it = container.end();   // итератор за последним элементом

++it;           // сдвиг вперёд
--it;           // сдвиг назад (для двухнаправленных)
*it;            // доступ к значению
it->member;     // доступ к члену
```

Алгоритмы

Алгоритмы — это функции для обработки последовательностей. Они работают через итераторы и не зависят от конкретного контейнера. Основные группы:

Немодифицирующие алгоритмы:

- `std::find` — поиск элемента;
- `std::count` — подсчёт элементов;
- `std::for_each` — применение функции к каждому элементу;
- `std::all_of`, `std::any_of`, `std::none_of` — проверка условий.

Модифицирующие алгоритмы:

- `std::copy` — копирование;
- `std::fill` — заполнение;
- `std::transform` — преобразование;
- `std::remove`, `std::remove_if` — удаление;
- `std::replace` — замена.

Сортировка и перестановки:

- `std::sort` — быстрая сортировка;
- `std::stable_sort` — устойчивая сортировка;
- `std::reverse` — обратный порядок;
- `std::rotate` — циклический сдвиг;

– `std::shuffle` — перемешивание.

Числовые алгоритмы:

– `std::accumulate` — сумма;

– `std::inner_product` — скалярное произведение;

– `std::partial_sum` — префиксные суммы.

Функторы

Функторы — это объекты, которые можно вызывать как функции. Они используются для настройки алгоритмов. Пример:

```
struct Greater {  
    bool operator()(int a, int b) const { return a > b; }  
};  
  
std::sort(v.begin(), v.end(), Greater());
```

С C++11 вместо функторов часто используются лямбда-выражения:

```
std::sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
```

Адаптеры

Адаптеры — это компоненты, изменяющие интерфейс других компонентов. Контейнерные адаптеры (`stack`, `queue`, `priority_queue`) предоставляют ограниченный интерфейс поверх других контейнеров. Итераторные адаптеры (`reverse_iterator`, `back_insert_iterator`) изменяют поведение итераторов. Функциональные адаптеры (`std::bind`, `std::not_fn`) изменяют поведение функторов.

Часть II. Двухнаправленный список `std::list`

Что такое двухнаправленный список

Двухнаправленный связный список — это структура данных, в которой каждый элемент (узел) содержит:

- само значение;
- указатель на предыдущий узел;
- указатель на следующий узел.

Благодаря этому можно эффективно перемещаться по списку в обоих направлениях и вставлять/удалять элементы в любом месте за константное время $O(1)$, если известна позиция. Однако доступ к элементу по индексу невозможен — чтобы добраться до n -го элемента, нужно пройти n шагов, что даёт $O(n)$.

Создание списка

Пустой список:

```
std::list<int> lst;
```

Список с заданным размером:

```
std::list<int> lst(5);    // 5 элементов, инициализированных нулями
```

```
std::list<int> lst(5, 42);    // 5 элементов, каждый равен 42
```

Инициализация списком инициализации:

```
std::list<int> lst = {1, 2, 3, 4, 5};
```

Копирование и перемещение:

```
std::list<int> copy = lst;      // копия
```

```
std::list<int> moved = std::move(lst); // перемещение
```

Список из диапазона:

```
std::vector<int> v = {10, 20, 30};
```

```
std::list<int> lst(v.begin(), v.end());
```

Список с пользовательским аллокатором:

```
std::list<int, std::allocator<int>> lst;
```

Добавление элементов

Метод `push_front(x)` — добавить элемент в начало.

Метод `push_back(x)` — добавить элемент в конец.

Метод `emplace_front(args...)` — сконструировать элемент в начале.

Метод `emplace_back(args...)` — сконструировать элемент в конце.

Метод `insert(pos, x)` — вставить элемент перед позицией `pos`.

Метод `insert(pos, n, x)` — вставить `n` копий `x`.

Метод `insert(pos, first, last)` — вставить диапазон.

Метод `emplace(pos, args...)` — сконструировать элемент перед `pos`.

Удаление элементов

Метод `pop_front()` — удалить первый элемент.

Метод `pop_back()` — удалить последний элемент.

Метод `erase(pos)` — удалить элемент по итератору.

Метод `erase(first, last)` — удалить диапазон.

Метод `remove(value)` — удалить все элементы со значением `value`.

Метод `remove_if(pred)` — удалить элементы, удовлетворяющие предикату.

Метод `clear()` — удалить все элементы.

Доступ к элементам

Метод `front()` — первый элемент.

Метод `back()` — последний элемент.

```
lst.front();
```

```
lst.back();
```

Обращение по индексу `lst[i]` невозможно — это принципиальное отличие от `std::vector` и `std::deque`.

Размер и проверка на пустоту

Метод `size()` — количество элементов.

Метод `empty()` — `true`, если список пуст.

Метод `max_size()` — максимально возможное количество элементов.

```
lst.size();
```

```
lst.empty();
```

Итераторы

`std::list` предоставляет двунаправленные итераторы (`bidirectional iterator`). Они поддерживают операции `++` и `--`, но не `+` и `-`.

Метод `begin()` / `end()` — прямые итераторы.

Метод `rbegin()` / `rend()` — обратные итераторы.

Метод `cbegin()` / `cend()` — константные прямые.

Метод `crbegin()` / `crend()` — константные обратные.

```
for (auto it = lst.begin(); it != lst.end(); ++it) {  
    std::cout << *it << ' '  
}  
  
for (auto it = lst.rbegin(); it != lst.rend(); ++it) {  
    std::cout << *it << ' '  
}
```

Для перемещения итератора на несколько позиций используется `std::advance`:

```
auto it = lst.begin();  
std::advance(it, 3);  
std::advance(it, -2);
```

Также есть `std::next` и `std::prev`:

```
auto it2 = std::next(it, 2);  
auto it3 = std::prev(it, 1);
```

Специфические методы `std::list`

Эти методы работают эффективнее общих алгоритмов STL, потому что используют структуру связанного списка.

Метод `sort()` — сортировка (по возрастанию).

Метод `sort(comp)` — сортировка с компаратором.

Метод `reverse()` — обратный порядок.

Метод `merge(other)` — слияние двух отсортированных списков.

Метод `merge(other, comp)` — слияние с компаратором.

Метод `splice(pos, other)` — переместить все элементы из `other` перед `pos`.

Метод `splice(pos, other, it)` — переместить один элемент.

Метод `splice(pos, other, first, last)` — переместить диапазон.

Метод `unique()` — удалить подряд идущие дубликаты.

Метод `unique(pred)` — удалить дубликаты по предикату.

Метод `assign`

Метод `assign(n, x)` — заменить содержимое `n` копиями `x`.

Метод `assign(first, last)` — заменить содержимое диапазоном.

Метод `assign({...})` — заменить содержимым списка инициализации

Алгоритмы STL, применимые к `std::list`

Многие алгоритмы STL работают с `std::list`, но не все эффективны. Например, `std::sort` не работает с `std::list`, потому что требует итераторов произвольного доступа. Вместо него используется `lst.sort()`.

Алгоритм `std::find` — применим, линейный поиск.

Алгоритм `std::count` — применим, подсчёт элементов.

Алгоритм `std::for_each` — применим, применение функции.

Алгоритм `std::copy` — применим, копирование.

Алгоритм `std::remove` — применим, но лучше `lst.remove()`.

Алгоритм `std::sort` — не применим, нужны итераторы произвольного доступа.

Алгоритм `std::reverse` — применим, но лучше `lst.reverse()`.

Алгоритм `std::merge` — применим, но лучше `lst.merge()`.

Алгоритм `std::unique` — применим, но лучше `lst.unique()`.

Пример использования `std::find`:

```
auto it = std::find(lst.begin(), lst.end(), 42);  
if (it != lst.end()) {  
    std::cout << "Found: " << *it << '\n';  
}
```

Когда использовать `std::list`

Плюсы:

- вставка и удаление в любом месте за $O(1)$ (при известной позиции);
- эффективное перемещение элементов между списками через `splice`;
- итераторы не инвалидируются при вставке/удалении других элементов (кроме удаляемого);
- эффективная работа с большими объёмами данных, где важна частая модификация середины.

Минусы:

- нет произвольного доступа по индексу;
- большой расход памяти: каждый узел хранит два указателя;
- плохая локальность кэша: элементы разбросаны по памяти;
- медленный обход по сравнению с `std::vector`.

Сравнение с другими контейнерами

`std::vector` — доступ по индексу $O(1)$, вставка/удаление в середине $O(n)$, память компактная.

`std::deque` — доступ по индексу $O(1)$, вставка/удаление в середине $O(n)$, память блочная.

`std::list` — доступа по индексу нет, вставка/удаление в середине $O(1)$, память узловая с указателями.

`std::forward_list` — доступа по индексу нет, вставка/удаление в середине $O(1)$, память односвязная, экономнее.

Пример: работа с двунаправленным списком

```
#include <iostream>

#include <list>

#include <algorithm>

int main() {

    std::list<int> lst = {5, 2, 8, 1, 9};

    lst.push_front(0);

    lst.push_back(10);

    lst.sort();

    std::cout << "Sorted list: ";

    for (int x : lst) std::cout << x << ' ';

    std::cout << '\n';

    lst.remove_if([](int x){ return x % 2 == 0; });

    std::cout << "After removing even: ";

    for (int x : lst) std::cout << x << ' ';

    std::cout << '\n';
```

```
    return 0;  
}
```